

SPM: SIMD programming on CPUs

Academic year 2025-2026

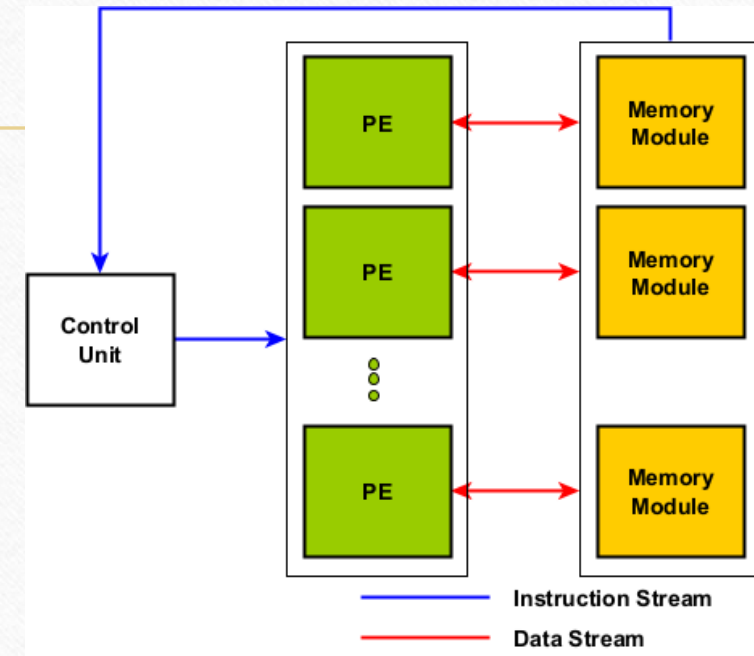
Prof. Massimo Torquati
massimo.torquati@unipi.it

Outline

- Explicit SIMD vectorization with Intrinsics
 - Low-level
- Automatic Compiler vectorization
 - High-level

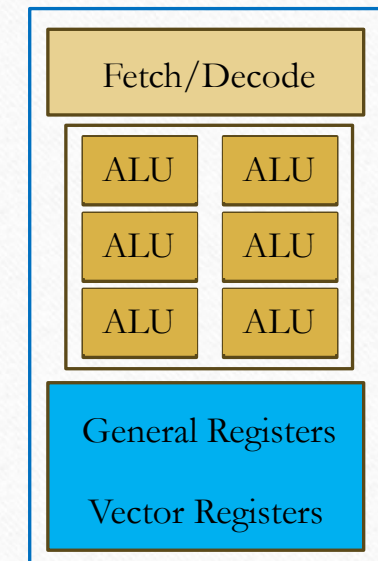
Why do we have to learn SIMD programming?

- It is a fundamental way to exploit data parallelism on modern CPUs
- Provides the foundation to understand and write efficient GPU kernels (CUDA/OpenCL/SYCL)
- SIMD can be the difference between achieving ~10% vs ~80% of a CPU's peak performance



Vector Units

- Modern CPU cores typically contain one or more **vector units** that can process multiple data elements in parallel
- The vector unit contains multiple ALUs (called **lanes**), each handling a different data element of a **vector register**
- Vector units include **vector registers** (also called SIMD registers) that are wider than standard scalar registers
 - Typical size: 128 bit (SSE), 256 bit (AVX/AVX2), 512 bit (AVX-512)
- A vector unit performs arithmetic/logic operations on several data elements (e.g., 8 floats) in one instruction (also called **vector instruction**) rather than processing them sequentially
 - Example: for 256-bit registers with float (32-bit), $256/32=8$ lanes; one instruction processes 8 data elements simultaneously
- In the SIMD model, the same instruction is broadcasted to all ALUs by the control unit
- Each ALU computes **the operation in parallel on different data**



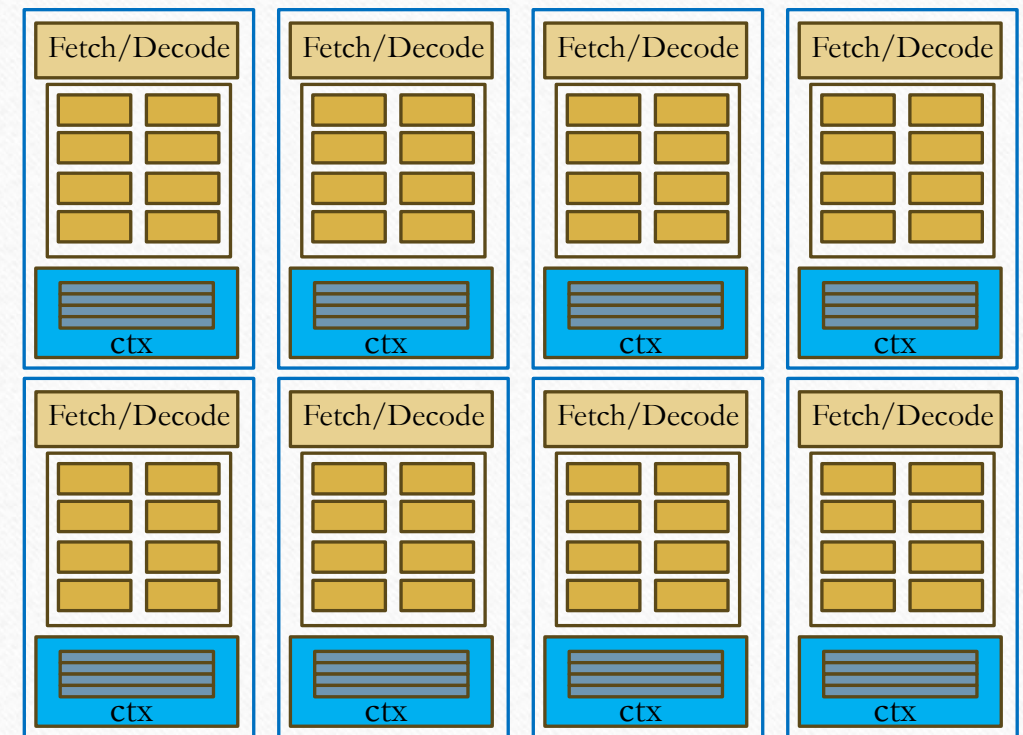
Vector operations

- x86-based cores support SIMD extensions such as **SSE** (128-bit vectors), **SSE2/SSE3/SSE4** (additional ops), and **Advanced Vector Extensions** (AVX/AVX2) (256-bit vectors), and **AVX-512** (512-bit vectors)
 - AVX2 introduced integer operations, gather instructions, and some enhancements for floating-point
 - The **FMA3** extension, added the **Fused Multiply-Add** operation on 128-bit and 256-bit
- ARM's **NEON** extension provides fixed 128-bit vectors, whereas **SVE** offers variable-length vector registers ranging from 128 to 2048 bits
- RISC-V Vector Extension (**RVV**) enables flexible vector lengths up to 2048 bits
- Compilers attempt to automatically apply vectorization techniques (**compiler vectorization**) to improve performance by using SIMD instructions
 - Programmers can manually optimize their code using **SIMD intrinsic instructions** when automatic vectorization is not enough

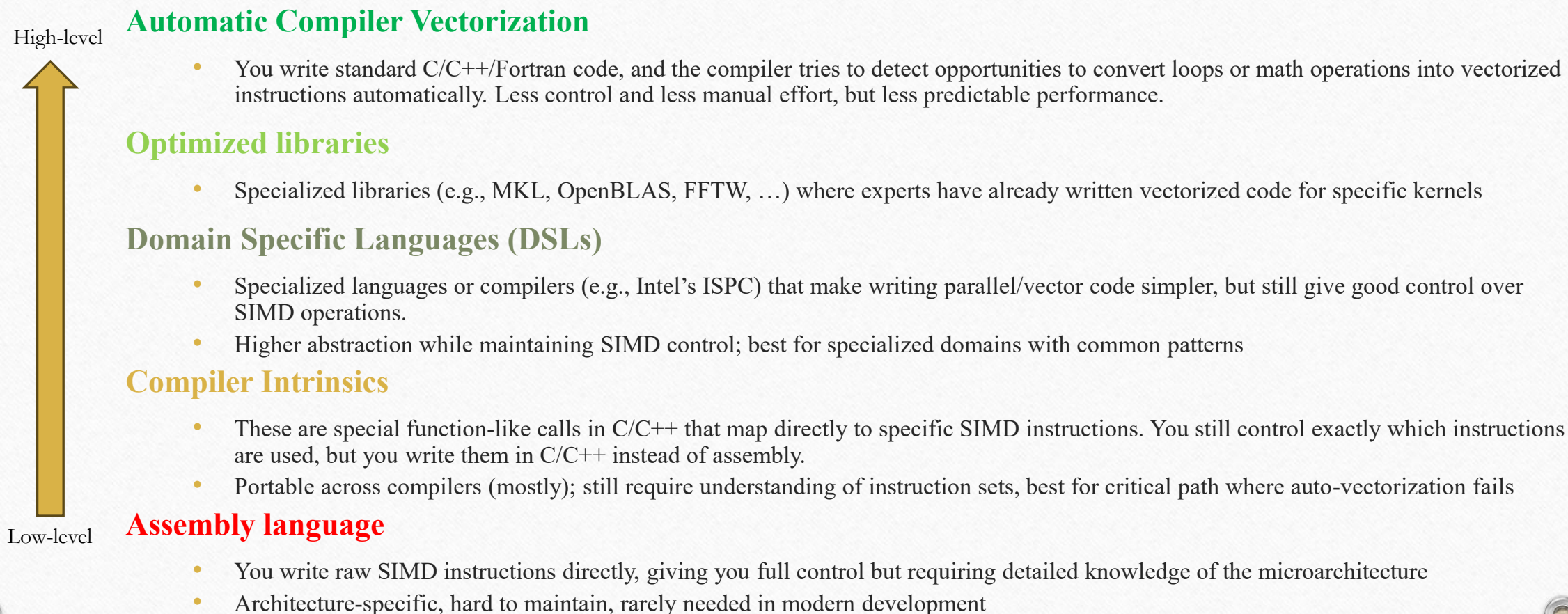
Flash Forward: on CUDA-enabled GPUs, threads within the so-called *warp* operate in a **SIMD lockstep fashion**

Multi-core with Vector Units

- Let's suppose a 8-core processor, each with one vector unit containing 8 lanes
- 64 lanes potentially working simultaneously on 8 instructions to compute 64 elements in parallel
- Modern high-end processors have 2-4 vector units per core
 - E.g., several modern Intel server cores can issue 2 AVX-512 FMA instructions
- Considering AVX-512, and float operations (FP32), and a CPU with 64 cores, each with 2 vector units, we can have up to 2048 float values processed in parallel
 - Note that this is a theoretical FLOP bandwidth; practical throughput depends on instruction scheduling, data movement, cache/memory bandwidth, and many other factors
- How to utilize such parallelism?



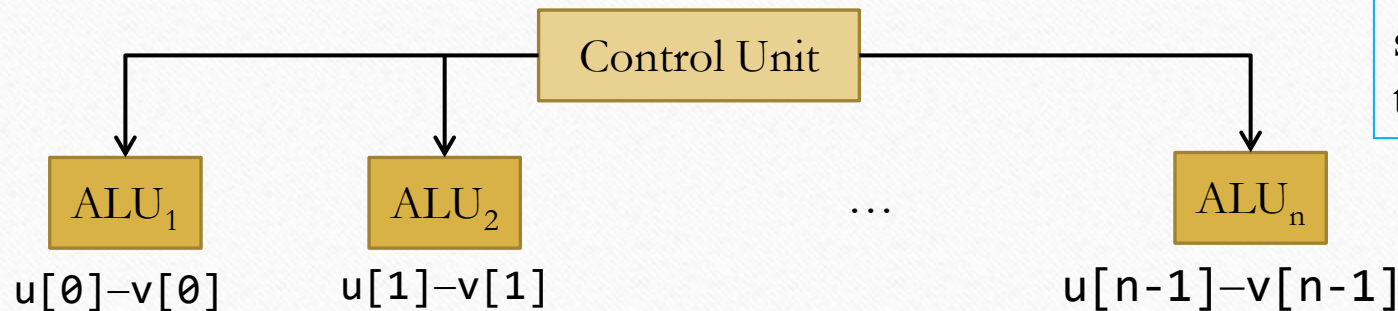
Programming Vector Units



Vector program

- All ALUs in a core execute the same instruction at any given clock cycle on a different set of data
- Synchronous execution (**lockstep execution**)

```
//Mapping element-wise subtraction onto SIMD  
for (int i = 0; i<n; i++)  
    w[i] = u[i]-v[i];
```



Performance considerations:

Performance improvement (speedup) is roughly $vector_width \times efficiency$
For an 8-wide float vector, the theoretical speedup is 8×, the actual speedup is typically lower, 4× to 6×

- What if we don't have as many ALUs as data items? **Divide the work and process it iteratively!**
- What if the array length is not multiple of the SIMD vector length? **Handle the remaining with scalar loop**

SIMD Intrinsics

- **What are Intrinsics?**

- Special compiler-supported functions that enable direct use of CPU-specific SIMD instructions
- Bridge between high-level C/C++ code and low-level assembly
- They enable access to low-level HW instructions while maintaining **portability across compilers on the same ISA** and ease of use compared to pure assembly
- Not all intrinsic functions map one-to-one to a single assembly instruction - some may be implemented using multiple instructions

- **Why using Intrinsics?**

- Intrinsics are expanded inline eliminating function call overhead
- Enable direct access to vectorized operations for higher performance
- More readable and maintainable than raw assembly while providing similar performance
- Allow programmers to manually optimize code where compiler auto-vectorization might fail

- **When not to use Intrinsics?**

- When standard library functions exist (e.g., MKL)
- When auto-vectorization works well enough

Programming with AVX2 Intrinsics

register size

```
#include <immintrin.h>    // Includes all Intel
                           // intrinsics (SSE to AVX-512)

__m256 a, b, c;           // declare registers
...                       // initialize a and b
c = _mm256_add_ps(a, b);  // c[0:8]=a[0:8]+b[0:8]
```

operation: add, sub, load, set, ...

packed single-precision floating point

`_mm<bit-width>_<operation>_<modifier>`

`<bit-width>`: 128, 256, 512

`<operation>`: add, sub, mul, load, ...

`<modifier>`:

- ps/pd: packed single/double precision
- epi32/64: packed 32/64-bit integers
- si128/256: 128/256-bit integers
- s: scalar (single element operation)
- p: packed (multiple elements)
- u: unaligned load/store

- Intel® Intrinsics Guide
<https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.htm>
- Intel® C++ Compiler Classic Developer Guide and Reference
<https://www.intel.com/content/www/us/en/docs/cpp-compiler/developer-guide-reference/2021-10/overview.html>
(Intrinsics section)

Examples:

```
_mm256_add_ps(a, b)    // 256-bit, add, packed singles (8 floats)
_mm256_mul_pd(a, b)    // 256-bit, multiply, packed doubles (4 doubles)
_mm256_loadu_ps(&a[i]) // 256-bit, load, unaligned, packed singles
_mm_set1_ps(5.0f)      // 128-bit, set all lanes to 5.0f
_mm_add_ss(a,b)         // (SSE): adds only the lowest float
                       // (scalar single-precision)
```


Example (AVX2)

```
// Adding two arrays using AVX intrinsics
```

```
#include <immintrin.h>
```

```
// For simplicity, assume len multiple of 8,
```

```
// and aligned memory (i.e., a,b, and c are aligned to 32 bytes)
```

```
void add_arrays_avx(const float* a, const float* b, float* c, long len) {
```

```
    for (long i = 0; i < len; i += 8) {
```

```
        __m256 va = _mm256_load_ps(&a[i]); // load 8 floats from a
```

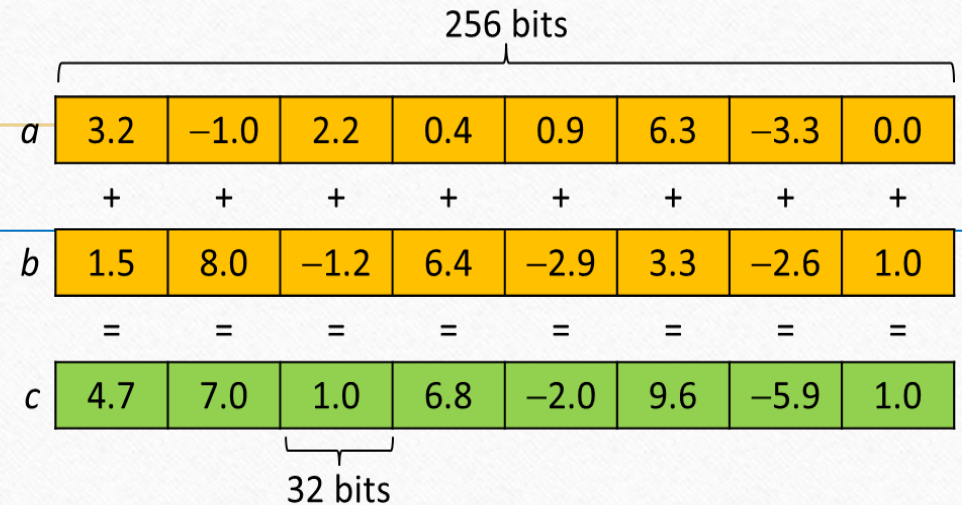
```
        __m256 vb = _mm256_load_ps(&b[i]); // load 8 floats from b
```

```
        __m256 vs = _mm256_add_ps(va, vb); // SIMD addition: vs[0:7]= va[0:7] + vb[0:7]
```

```
        _mm256_store_ps(&c[i], vs); // store 8 results to c
```

```
    }
```

```
}
```



Aligned and Unaligned operations

- Aligned and unaligned operations refer to how data is stored in memory relative to certain byte boundaries
- Data is aligned when its memory address is a multiple of a specific boundary (e.g., 32 bytes for AVX)
 - Aligned memory accesses are generally faster. For unaligned accesses, the CPU may need to perform extra work to handle data crossing cache line boundaries
 - Unaligned is often fine unless it crosses cache-line boundaries
- Why are both necessary?
 - Not all data structures are naturally aligned, e.g., those with complex layouts.
 - Programmers need to evaluate the convenience/flexibility of unaligned operations vs. performance gains
- Rules
 - Prefer aligned allocation when you control the allocation. Typical alignments: SSE 16B, AVX 32B, AVX-512 64B

How to allocate and verify aligned memory

```
// C++17
float *a = new(std::align_val_t(32)) float[size];
delete [] a;
```

```
// C11, it requires size to be a multiple
// of the alignment
float *a=(float*)aligned_alloc(32, sizeByte);
free(a);
```

```
// intrinsics
float *a=(float*)_mm_malloc(sizeByte, 32);
_mm_free(a);
```

```
// Check alignment
bool is_aligned(const void* p, size_t algn) {
    return(reinterpret_cast<std::uintptr_t>(p) % algn) == 0;
}
```

```
// AVX example
float* a = static_cast<float*>(aligned_alloc(32, 1024*sizeof(float)));
assert(is_aligned(a, 32));
float* b = new float[1024];

__m256 v1 = _mm256_load_ps(a); // aligned load, a must be 32-byte aligned
__m256 v2 = _mm256_loadu_ps(b); // unaligned load, safe for any address
```

Fused Multiply Add (FMA)

- FMA is a single instruction that performs a multiplication and an addition
- Why use FMA?

$$\text{FMA}(\mathbf{A}, \mathbf{B}, \mathbf{C}) = (\mathbf{A} \times \mathbf{B}) + \mathbf{C}$$

- Performance

- Single instruction instead of two (MUL + ADD)
- Often executes in same cycles as separate MUL or ADD
- Reduced instruction count → better throughput
- Critical for matrix operations, dot products, polynomials

- HW support

- Dedicated FMA units in modern CPUs and GPUs
- Can often issue 2 FMA ops per cycle per core
- Example: AVX-512 FMA unit can do 16 FMA operations (32 FLOPs) per cycle

- Numerical accuracy

- Only one rounding operation (at the end) instead of two rounding operations for separate MUL + ADD
- Reduced floating-point error accumulation

```
_mm256_fmadd_ps(AV, BV, X);
```

AV	A[i*L]	A[i*L+1]	A[i*L+2]	A[i*L+3]	A[i*L+4]	A[i*L+5]	A[i*L+6]	A[i*L+7]
	*	*	*	*	*	*	*	*
BV	B[j*L]	B[j*L+1]	B[j*L+2]	B[j*L+3]	B[j*L+4]	B[j*L+5]	B[j*L+6]	B[j*L+7]
	+	+	+	+	+	+	+	+
X	X[0]	X[1]	X[2]	X[3]	X[4]	X[5]	X[6]	X[7]
	=	=	=	=	=	=	=	=
X	X[0]	X[1]	X[2]	X[3]	X[4]	X[5]	X[6]	X[7]

Examples:

```
_mm256_fmadd_ps(a, b, c); // c + (a × b)
_mm256_fmsub_ps(a, b, c); // (a × b) - c
_mm256_fnmadd_ps(a, b, c); // c - (a × b)
_mm256_fnmsub_ps(a, b, c); // -(a × b) - c
```


Textbook GEMM with AVX2

```
// Textbook GEMM with AVX2 and FMA
// A=N x L, B=L x M, C=N x M (assume M multiple of 8)
void mm_avx(float * A, float * B, float * C,
            uint64_t M, uint64_t L, uint64_t N) {
    for (uint64_t i=0; i<N; i++) // rows of A
        for (uint64_t j=0; j<M; j+=8) { // cols of B (8 at a time)
            // init temporary vector X
            __m256 X = _mm256_setzero_ps(); // X=[0,0,0,0,0,0,0,0]
            for (uint64_t k=0; k<L; k++) {
                // replicate A[i][k] across all 8 lanes
                __m256 Aik = _mm256_broadcast_ss(&A[i*L+k]);
                // Aik = [A[i][k], A[i][k], ..., A[i][k]]
                // load 8 contiguous float from B[k][j:j+7]
                __m256 BV = _mm256_load_ps(&B[k*M+j]);
                // BV = [B[k][j], B[k][j+1], ..., B[k][j+7]]
                // X += Aik x BV (8 FMAs in one instruction)
                X = _mm256_fmadd_ps(Aik, BV, X);
            }
            // store the computed 8 values into C[i][j:j+7]
            _mm256_store_ps(&C[i*M+j], X);
        }
}
```

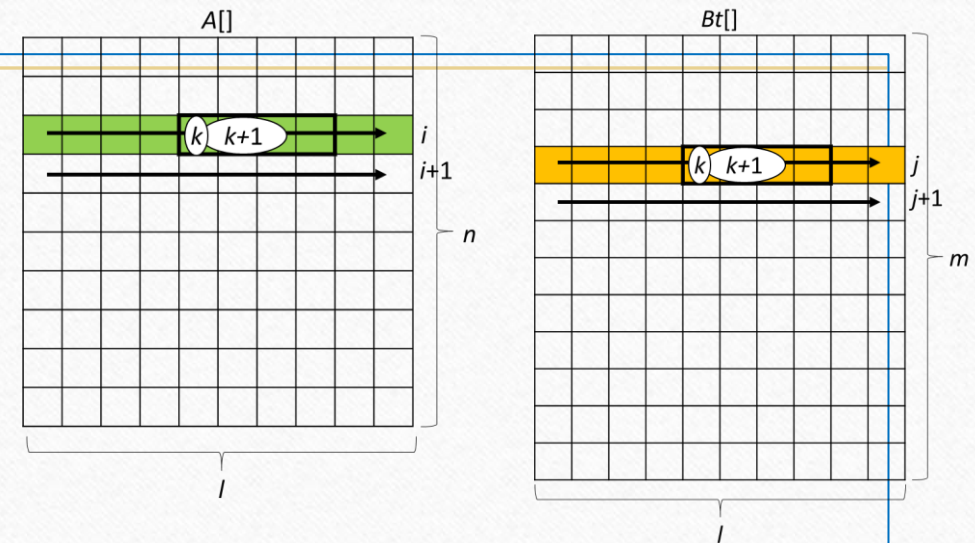
```
[torquati@spmln spmcode2]$ ./mm_avx 13 13 8
# elapsed time (alloc_memory): 2.3146e-05s
# elapsed time (init): 0.0210694s
# elapsed time (naive_mm): 36.4571s
# elapsed time (naive_mm_avx): 4.76097s
```

SPM cluster (front-end node) **7.6x speedup**
Intel(R) Xeon(R) CPU **94% efficiency**
E5-2650 v3 @ 2.30GHz

- Processes a block of 8 contiguous C elements at a time: $C[i][j..j+7]$ (vectorization across j)
- 8 FMA operations for each k-loop iteration
- Each FMA = $2 \text{ FLOP} \times 8 \text{ lanes} = 16 \text{ FLOP}$
- Total FLOP:
$$N \times (M/8) \times L \times 16 = 2 \times M \times L \times N$$

Transposed GEMM with AVX2

```
// Transpose-and-Multiply with AVX2 and FMA
// Here we assume B already transposed
void avx_tmm(float * A, float * B, float * C,
            uint64_t M, uint64_t L, uint64_t N) {
    for (uint64_t i=0; i<M; i++)
        for (uint64_t j=0; j<N; j++) {
            __m256 X = _mm256_setzero_ps(); // Accumulator
            for (uint64_t k=0; k<L; k+=8) { // 8 at a time
                // Load 8 elements from A[i][k:k+7]
                const __m256 AV = _mm256_load_ps(A+i*L+k);
                // Load 8 elements from B[j][k:k+7]
                const __m256 BV = _mm256_load_ps(B+j*L+k);
                X = _mm256_fmadd_ps(AV, BV, X); // X += AV x BV (element-wise MUL and SUM)
            }
            // X now contains 8 partial sums that need to be added together
            C[i*N+j] = hsum_avx(X); // horizontal sum reduction; from 8 floats to a single one
        }
}
```



implementation in the next slide

hsum_avx

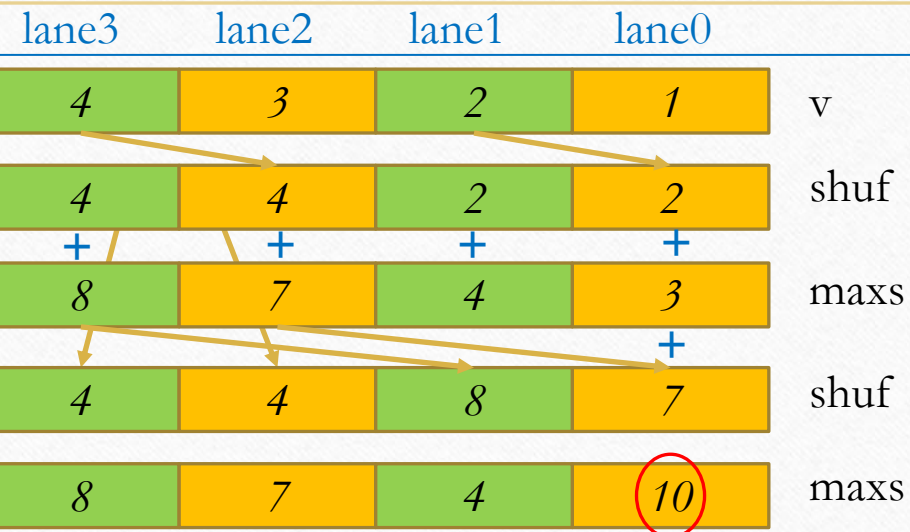
Strategy:

1. Split 256-bit into two 128-bit halves (lo and hi)
2. Add the two halves together (lo + hi → lo, 4 values)
3. Use SSE horizontal sum on the 4 values

```
// horizontal sum using SSE3 (4 floats)
```

```
inline float hsum_sse3(__m128 v) {  
    __m128 shuf = _mm_movehdup_ps(v);  
    __m128 maxs = _mm_add_ps(v, shuf);  
    shuf = _mm_movehl_ps(shuf, maxs);  
    maxs = _mm_add_ss(maxs, shuf);  
    // extract the lowest 32-bit float  
    return _mm_cvtss_f32(maxs);  
}
```

```
inline float hsum_avx(__m256 v) {  
    __m128 lo = _mm256_castps256_ps128(v); // cast the lower 128-bits of v (no move)  
    __m128 hi = _mm256_extractf128_ps(v, 1); // moves the higher 128-bits of v  
    lo = _mm_add_ps(lo, hi); // adds the four float values of lo and hi  
    return hsum_sse3(lo); // horizontal sum of 4 floats  
}
```



Transposed AVX2 GEMM performance

Execution on an i7-6800K using $m = 1K$, $l = 2K$, $n = 4K$

#elapsed time (plain_tmm): 12.29s

#elapsed time (avx2_tmm): 2.13s

} Speedup: 5.8

SPM cluster (front-end node)
Intel(R) Xeon(R) CPU
E5-2650 v3 @ 2.30GHz

```
[torquati@spmln spmcode2]$ ./tmm_avx 13 13 8
# elapsed time (alloc_memory): 2.5701e-05s
# elapsed time (init): 0.0208098s
# elapsed time (plain_tmm): 16.9971s
# elapsed time (avx_tmm): 3.22379s
```

} Speedup: 5.27

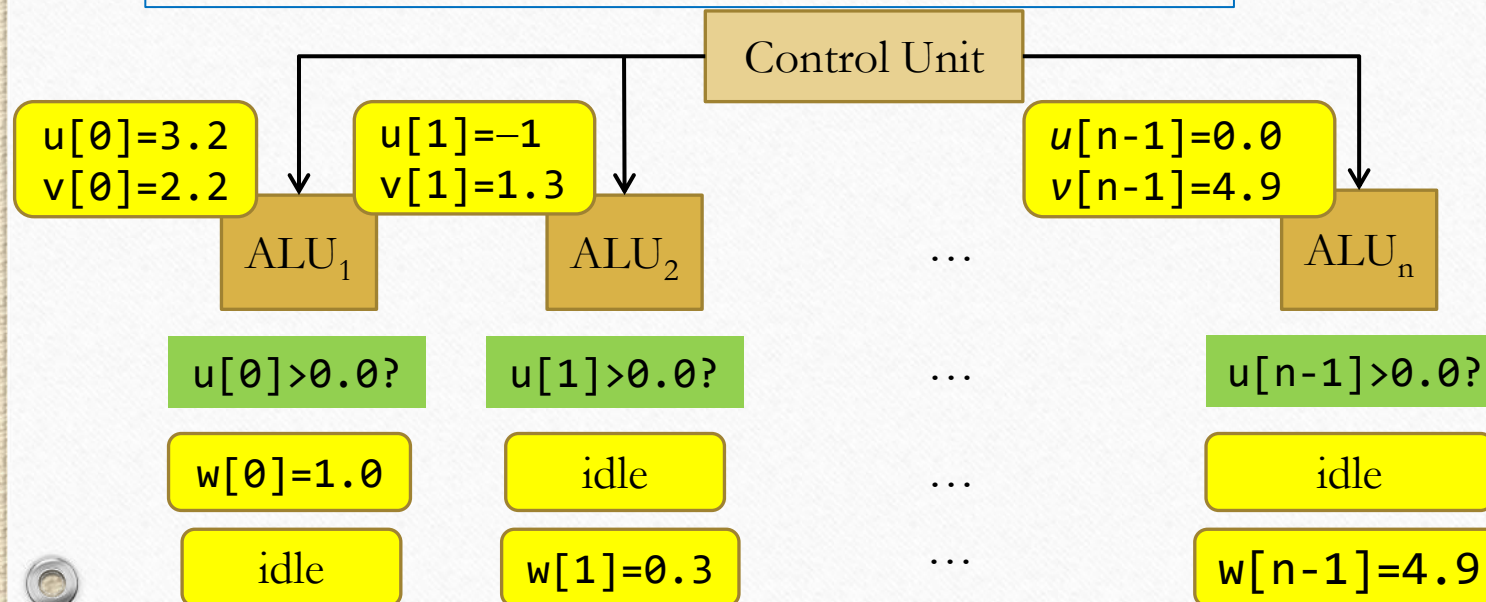
Divergence concept

//Mapping a conditional Statement onto SIMD

```
for (i = 0; i<n; i++)  
  if (u[i] > 0)  
    w[i] = u[i]-v[i];  
  else  
    w[i] = u[i]+v[i];
```

Divergent branching:

When lanes evaluate a condition individually and take different control paths due to branches



- All ALUs must execute the same instruction (synchronously) or idle
- Many SIMD processors handle divergence by serializing the execution of different branches
- This problem can be addressed using **predication**

Predication

- **What can we do to handle divergence?**
 - Instead of branching, compute both paths and select results: in this example, we must compute both the multiply by 2 (mul) and the division by 2 (div)
 - Create a mask of elements with all bits set to 1 if the computed value is less than 0.0f, and 0 otherwise
 - Use the mask to select the results (blending) from mul or from div
 - This way, all lanes (elements) go through the same sequence of instructions (there is no if statement)

```
// Scalar version with branching
void transform_scalar(const float* in,
                     float* out,
                     size_t n) {
    for (size_t i = 0; i < n; i++) {
        if (in[i] < 0.0f)
            out[i] = in[i] * 2.0f;
        else
            out[i] = in[i] / 2.0f;
    }
}
```

in	-1	1	-2	-1	0	1	1	-3
path1	-2	2	-4	-2	0	2	2	-6
path2	-0.5	0.5	-1	-0.5	0	0.5	0.5	-1.5
mask	1	0	1	1	0	0	0	1
blend	-2	0.5	-4	-2	0	0.5	0.5	-6

Predication: Example 1

```
// Scalar version with branching
void transform_scalar(const float* in,
                    const float* out,
                    size_t n) {
    for (size_t i = 0; i < n; i++) {
        if (in[i] < 0.0f)
            out[i] = in[i] * 2.0f;
        else
            out[i] = in[i] / 2.0f;
    }
}
```



```
// AVX version using predication (branch-free)
void transform_scalar(const float* in, float* out, size_t n) {
    __m256 zero = _mm256_set1_ps(0.0f);
    __m256 two  = _mm256_set1_ps(2.0f);
    // suppose n % 8 = 0 for simplicity
    for (size_t i = 0; i < n; i += 8) {
        // unaligned load
        __m256 v      = _mm256_loadu_ps(&in[i]);
        __m256 mul     = _mm256_mul_ps(v, two);
        __m256 div     = _mm256_div_ps(v, two);
        // create the mask considering elements LT zero
        __m256 mask    = _mm256_cmp_ps(v, zero, _CMP_LT_OS);
        // if mask=1 take from mul otherwise take from div
        __m256 blend   = _mm256_blendv_ps(div, mul, mask);
        // store the results back to memory
        _mm256_storeu_ps(&out[i], blend);
    }
}
```

Predication: Example 2

```
// Scalar version with branching
void transform_scalar(const float* in,
                    float* out,
                    size_t n) {
    for (size_t i = 0; i < n; i++) {
        float x = in[i];
        float y;
        if (x < -1.0f) y = -1.0f;
        else if (x <= 1.0f) y = x * x;
        else y = 1.0f;
        out[i] = y;
    }
}
```



```
// AVX version using predication (branch-free)
void transform_scalar(const float* in, float* out, size_t n) {
    const __m256 v_m1 = _mm256_set1_ps(-1.0f);
    const __m256 v_p1 = _mm256_set1_ps( 1.0f);
    for (size_t i = 0; i < n; i += 8) { // suppose n % 8 = 0
        __m256 x = _mm256_loadu_ps(in + i);
        // masks
        __m256 m_l = _mm256_cmp_ps(x, v_m1, _CMP_LT_OQ); // x < -1
        __m256 m_h = _mm256_cmp_ps(x, v_p1, _CMP_GT_OQ); // x > 1
        // m_m is the complement of (low OR high)
        __m256 m_m = _mm256_andnot_ps(_mm256_or_ps(m_l, m_h),
                                       _mm256_castsi256_ps(_mm256_set1_epi32(-1)));

        // paths
        __m256 y_l = v_m1;
        __m256 y_m = _mm256_mul_ps(x, x);
        __m256 y_h = v_p1;
        // blend with priority. In this example, order doesn't matter
        __m256 y = y_m;
        y = _mm256_or_ps(_mm256_and_ps(m_l, y_l), _mm256_andnot_ps(m_l, y));
        y = _mm256_or_ps(_mm256_and_ps(m_h, y_h), _mm256_andnot_ps(m_h, y));
        _mm256_storeu_ps(out + i, y);
    }
}
```


Impact of branching

Example 1

Execution on spmcluster 10M elements

#elapsed time (scalar):	0.058s
#elapsed time (avx2_branch):	0.0471s
#elapsed time (avx2_pred):	0.0071s

Why is the speedup so good (**superlinear**) if compared to the scalar version?

- No branch mispredictions in AVX version
- All lanes always active (no idle time)
- Better instruction pipeline utilization
- Compiler can optimize better without branches

Example 2

Execution on spmcluster 10M elements

#elapsed time (scalar):	0.068s
#elapsed time (avx2):	0.0072s

Speedup: 9.44

Key takeaway: Branch-free, mask-based code is crucial for SIMD because it removes control-flow divergence and often reduces misprediction stalls, even if it may do extra work

AoS and SoA layouts

- To exploit the power of SIMD parallelism, it is often necessary to **modify the layout of the data structures used**.



```
//AoS-like layout  
auto xyz = new float[3*n];
```

- The case study (in the pictures) is a collection of n values representing 3D coordinates
- Two different layouts:
 - AoS (Array of Structures)**: stores records consecutively in a single array
 - SoA (Structure of Arrays)**: uses one array per dimension. Each array only stores the values of the associated element dimension



```
//SoA layout  
auto x = new float[n];  
auto y = new float[n];  
auto z = new float[n];
```


Vector Normalization with AoS

AoS

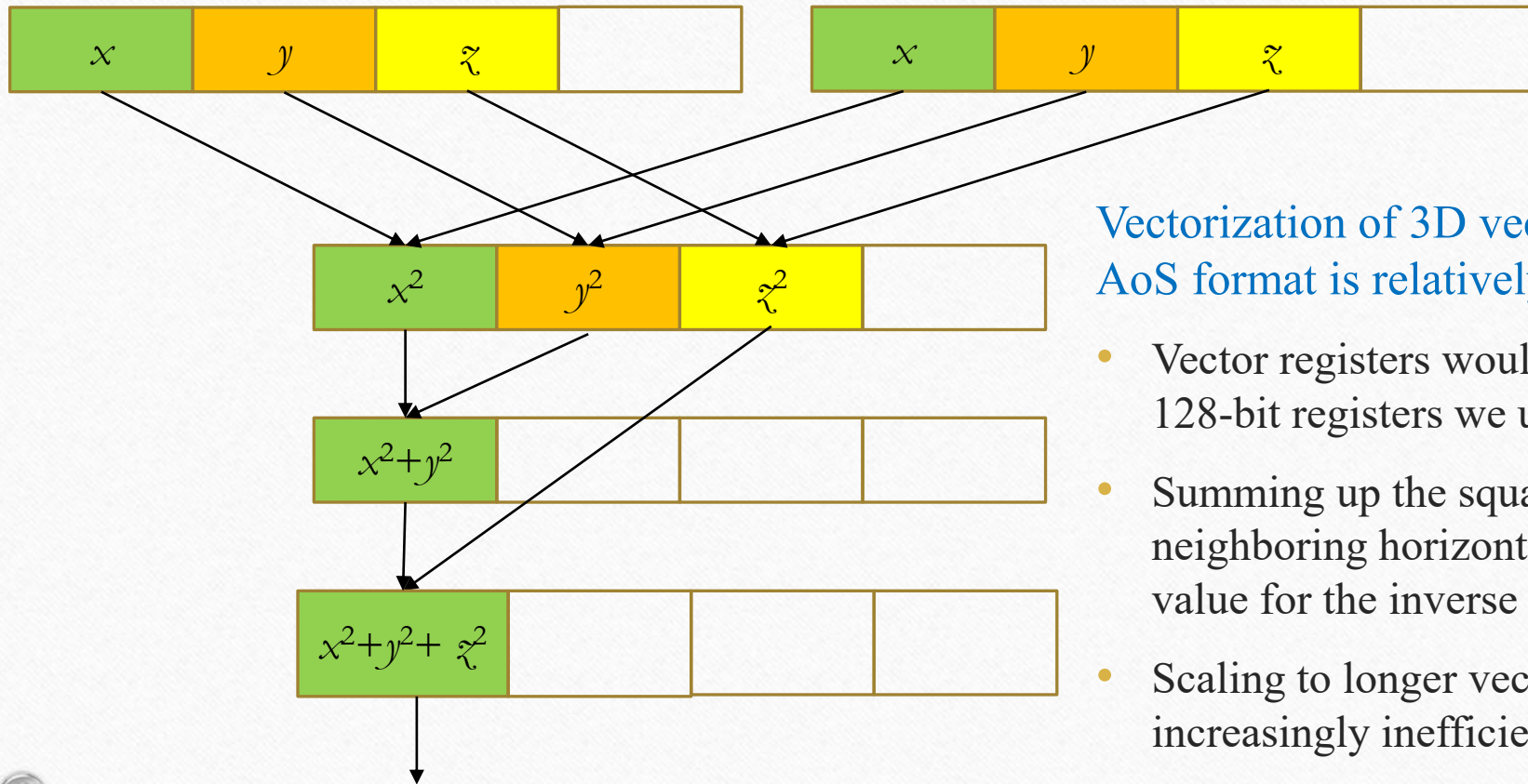
xyz[]

x ₀	y ₀	z ₀	x ₁	y ₁	z ₁	x ₂	y ₂	z ₂	x ₃	y ₃	z ₃	x ₄	y ₄	z ₄	x ₅	y ₅	z ₅	x ₆	y ₆	z ₆	x ₇	y ₇	z ₇
----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------

```
// Non-vectorized with plain AoS layout 3D vector normalization
void plain_aos_norm(float * xyz, uint64_t length) {
    for (uint64_t i=0; i<3*length; i+=3) {
        const float x = xyz[i+0];
        const float y = xyz[i+1];
        const float z = xyz[i+2];
        float irho = 1.0f/std::sqrt(x*x + y*y + z*z); // compute magnitude
        // normalize
        xyz[i+0] *= irho;
        xyz[i+1] *= irho;
        xyz[i+2] *= irho;
    }
}
```

Map each $v_i = (x_i, y_i, z_i)$ to $\hat{v}_i = \frac{v_i}{\|v_i\|} = \left(\frac{x_i}{\rho_i}, \frac{y_i}{\rho_i}, \frac{z_i}{\rho_i}\right)$ where $\rho_i = \sqrt{x_i^2 + y_i^2 + z_i^2}$

Vector Normalization with AoS (cont.)



Vectorization of 3D vector normalization based on the AoS format is relatively inefficient:

- Vector registers would not be fully occupied. E.g., for 128-bit registers we use three out of four vector lanes
- Summing up the squares requires operations between neighboring horizontal lanes, resulting in only a single value for the inverse square root calculation
- Scaling to longer vector registers becomes increasingly inefficient

Vector Normalization with SoA (cont.)

```
// AVX-Vectorized SoA layout 3D vector normalization
void avx_soa_norm(float *x, float *y, float *z, uint64_t length) {
    for (uint64_t i=0; i<length; i+=8) { // Process 8 vectors at once!
        __m256 X = _mm256_load_ps(x+i); // aligned loads
        __m256 Y = _mm256_load_ps(y+i); // [y0,y1,y2,y3,y4,y5,y6,y7]
        __m256 Z = _mm256_load_ps(z+i);
        // Compute  $\rho^2 = x^2 + y^2 + z^2$  (8 values at once!)
        __m256 R = _mm256_fmadd_ps(X,X, //  $X^2$ 
                                   _mm256_fmadd_ps(Y,Y, //  $+ Y^2$ 
                                   _mm256_mul_ps(Z,Z) ) ); //  $+ Z^2$ 
        //  $R = [\rho_0^2, \rho_1^2, \rho_2^2, \rho_3^2, \rho_4^2, \rho_5^2, \rho_6^2, \rho_7^2]$ 
        R = _mm256_rsqrt_ps(R); //  $R \leftarrow 1/\sqrt{R}$ 

        _mm256_store_ps(x+i, _mm256_mul_ps(X, R)); // aligned stores
        _mm256_store_ps(y+i, _mm256_mul_ps(Y, R)); //  $y/\rho$ 
        _mm256_store_ps(z+i, _mm256_mul_ps(Z, R));
    }
}
```

$$v_i = (x_i, y_i, z_i) \text{ to } \hat{v}_i = \left(\frac{x_i}{\rho_i}, \frac{y_i}{\rho_i}, \frac{z_i}{\rho_i} \right)$$

$$\text{where } \rho_i = \sqrt{x_i^2 + y_i^2 + z_i^2}$$

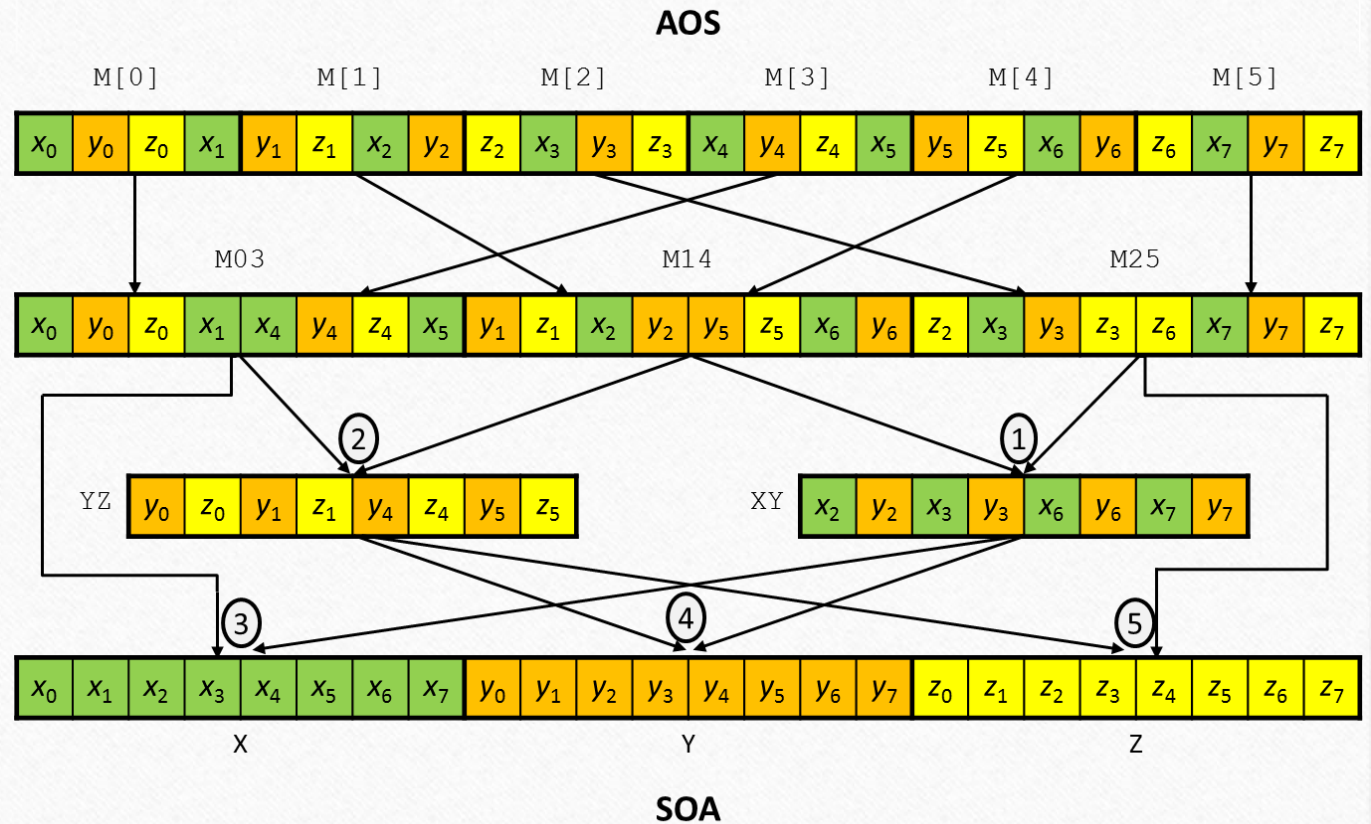
SoA

x []	x ₀	x ₁	x ₂	x ₃	x ₄	x ₅	x ₆	x ₇
y []	y ₀	y ₁	y ₂	y ₃	y ₄	y ₅	y ₆	y ₇
z []	z ₀	z ₁	z ₂	z ₃	z ₄	z ₅	z ₆	z ₇

In this version, during each loop iteration, 8 vectors are normalized simultaneously

From AoS to SoA using Vectorized Shuffling

- **Problem:** What if data is stored in AoS but we need SoA for computation?
 - Option 1:
Convert entire array (slow for large data)
 - Option 2:
Convert on-the-fly in chunks (shown here)



From AoS to SoA using Vectorized Shuffling

```
//AoS-to-SoA:   XYZXYZXY ZXYZXYZX YZXYZXYZ --> XXXXXXXX YYYYYYYY ZZZZZZZZ
```

```
__m128 *M = (__m128*) (xyz+i); // cast to an SSE pointer (length 4)
```

```
__m256 M03, M14, M25;
```

```
// load lower halves
```

```
M03 = _mm256_castps128_ps256(M[0]);
```

```
M14 = _mm256_castps128_ps256(M[1]);
```

```
M25 = _mm256_castps128_ps256(M[2]);
```

```
// load upper halves
```

```
M03 = _mm256_insertf128_ps(M03, M[3], 1);
```

```
M14 = _mm256_insertf128_ps(M14, M[4], 1);
```

```
M25 = _mm256_insertf128_ps(M25, M[5], 1);
```

```
// shuffling...
```

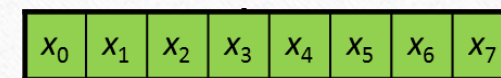
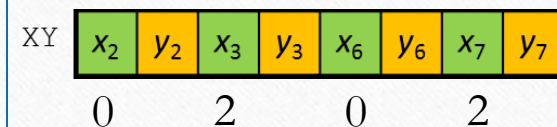
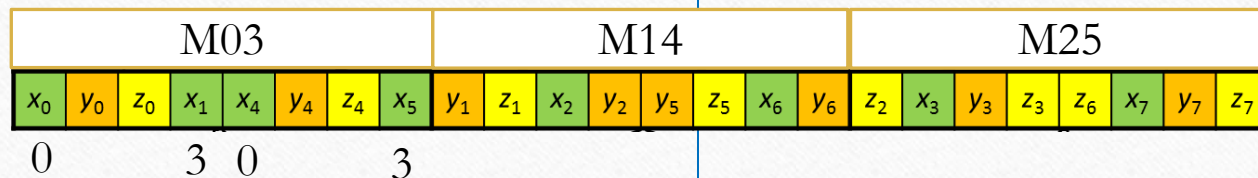
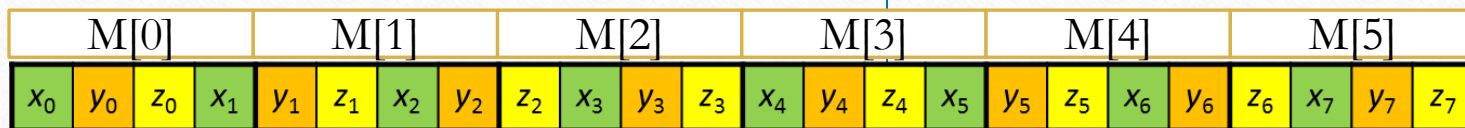
```
__m256 XY = _mm256_shuffle_ps(M14, M25, _MM_SHUFFLE( 2,1,3,2));
```

```
__m256 YZ = _mm256_shuffle_ps(M03, M14, _MM_SHUFFLE( 1,0,2,1));
```

```
__m256 X  = _mm256_shuffle_ps(M03, XY, _MM_SHUFFLE( 2,0,3,0));
```

```
__m256 Y  = _mm256_shuffle_ps(YZ, XY, _MM_SHUFFLE( 3,1,2,0));
```

```
__m256 Z  = _mm256_shuffle_ps(YZ, M25, _MM_SHUFFLE( 3,0,3,1));
```



AoS on the fly transposition into SoA and inverse transposition of results

- AoS may still be preferred for compact storage and because some operations benefit from tightly packed vectors
- To exploit SIMD anyway, we transpose on the fly from AoS to SoA on blocks of 8 consecutive 3D vectors (using three 256-bit registers)
- We run the vectorized computation in SoA layout, then transpose the results back from SoA to AoS.

Execution on an i7-6800K using $n = 2^{28}$

#elapsed time (plain_aos_normalize): 0.72s

#elapsed time (avx_aos_normalize): 0.33s

} Speedup: 2.2 (despite transposition)

Data layout best practice

- **First choice:** design with SoA from the start
 - Fastest, no transpose needed
 - Best SIMD efficiency and cache locality
 - Scale well with wider SIMD (AVX-512)
- **Second choice:** batch convert AoS into SoA
 - Convert large chunks/tiles at a time (e.g., 1K to 64K elements into a temporary SoA buffer
 - Run one or more SIMD kernel on the SoA chunk
 - Amortize conversion cost over multiple operations and keep the working set cache-friendly
- **Third choice:** on-the-fly transpose (as seen)
 - Useful when source data must remain AoS.
 - When memory is limited, convert a micro-tile on-the-fly
- **Last resort:** work directly with AoS
 - Use scatter/gather instructions if available (AVX2+)

```
[torquati@spmln spmcode]$ ./vector_norm_aos_plain 28
# elapsed time (init): 8.11706s
# elapsed time (plain_aos_normalize): 1.49564s
# elapsed time (check): 0.697728s
[torquati@spmln spmcode]$ ./vector_norm_soa_plain 28
# elapsed time (init): 8.04905s
# elapsed time (plain_soa_normalize): 1.53434s
# elapsed time (check): 0.664117s
# elapsed time (free_memory): 0.00727703s
[torquati@spmln spmcode]$ ./vector_norm_soa_avx 28
# elapsed time (init): 3.89911s
# elapsed time (avx_soa_normalize): 0.332816s
# elapsed time (check): 0.663632s
[torquati@spmln spmcode]$ ./vector_norm_aos_avx 28
# elapsed time (init): 3.95525s
# elapsed time (avx_aos_normalize): 0.407664s
# elapsed time (check): 0.69664s
```

Auto-vectorization

- What is auto-vectorization? A **compiler optimization that automatically transforms scalar loops into SIMD instructions**
 - No code change needed, compiler does the work
 - Aims to exploit wide vector registers (e.g., AVX/AVX-512) for improving performance
 - Success depends on loop characteristics and compiler capabilities

Auto-vectorization, some compiler's options

- Key GCC compiler flags:
 - `-O3`: enables aggressive optimizations including auto-vectorization (`-O2` in recent GCC versions also enables it)
 - `-march=native`: enables all instruction-set extensions available on your local CPU (e.g., SSE3, AVX2, ...)
 - Without this option, GCC generates generic instructions to ensure broad compatibility (e.g., only SSE2)
 - `-ftree-vectorize`: explicitly enables vectorization optimizations (enabled by `-O3`)
 - `-fopt-info-<option>`: enables optimization dumps (reports) about why the compiler did or did not vectorize
 - `-fopt-info-vec-all`: prints reports for all vectorization optimizations
 - `-fopt-info-vec-missed`: prints only the missed opportunity
 - `-ffast-math`: loosens strict IEEE 754 floating-point rules (**NOTE: may change numerical behavior**)
 - `-fprofile-generate` / `-fprofile-use`: Profile-Guided Optimizations (PGO) the compiler makes informed auto-vectorization decisions based on runtime data

More info: [Developer Options \(Using the GNU Compiler Collection \(GCC\)\)](#)

Enabling auto-vectorization, examples

```
// Basic auto-vectorization enabled
g++ -std=c++17 -O3 -march=native prog.cpp -o prog
```

Auto-vectorization, the compiler does not produce any report

```
// with diagnostics
g++ -std=c++17 -O3 -march=native -fopt-info-vec-all prog.cpp -o prog
```

Verbose report

We can have a report
on the loop missed

```
// Only loop vectorized and loop where vectorization failed
g++ -std=c++17 -O3 -march=native -fopt-info-vec-optimized -fopt-inf-vec-missed prog.cpp -o prog
```

```
// aggressive (using -ffast-math)
g++ -std=c++17 -O3 -march=native -ffast-math prog.cpp -o prog
```

-ffast-math allows FP reassociation and may
enable vector reductions, but can change results

```
// Disabling auto-vectorization
g++ -std=c++17 -O3 -march=native -fno-tree-vectorize prog.cpp -o prog
```

Selectively disable vectorization

```
// aggressive with maximum diagnostics, redirecting the output to a file
g++ -std=c++17 -O3 -march=native -ffast-math \
    -fopt-info-vec-all=vec.txt \
    -fopt-info-loop-all prog.cpp -o prog
```


Tips & Caveats for auto-vectorization

- Not all loops will be vectorized automatically; compilers can be conservative in many cases

- Why does it fail?

- Pointer aliasing, the compiler cannot assume ptrA and ptrB do not overlap
- Loop-carried dependencies (e.g., prefix sum)
- Non-unit stride or unknown alignment
- Function calls inside the loop, non-inlinable, or side effects

```
// prefix sum of vector in into out
float s = 0.0f;
for (auto i = 0; i < N; i++) {
    s += in[i];
    out[i] = s;
}
```

- How to help the compiler to vectorize the loop

- Use `__restrict__` to assert no aliasing, and also `const` to make loop bounds simple
- Align data, or use `__builtin_assume_aligned` hints if it holds true
- Use pragmas like `#pragma GCC ivdep` (or with OpenMP `#pragma omp simd`) only if you are sure
 - `ivdep` stands for “ignore vector dependencies”

```
#pragma GCC ivdep
for (auto i = 0; i < N; i++) {
    // loop body that you claim has
    // no dependencies
}
```

Loop iteration dependencies

- Types of dependencies that may prevent vectorization:

- True dependencies (**Read-after-Write - RAW**)

- An iteration uses data produced in a previous iteration

```
for (int i=1; i<N; ++i)
    A[i] = A[i-1] + B[i];
```

- Anti dependencies (**Write-after-Read - WAR**)

- An iteration writes to a memory location read in a previous iteration and written in a later iteration

```
for (int i=1; i<N-1; ++i) {
    B[i] = A[i];
    A[i-1] = 0;
}
```

- Output dependencies (**Write-after-Write - WAW**)

- Multiple iterations write to the same memory location (write conflicts across iterations)

```
for (int i=1; i<N; ++i)
    A[i % 10] = foo(i);
```

- With these dependencies, the compiler does not automatically vectorize the loop** because it cannot prove it can reorder or execute iterations in parallel without changing semantics

Vectorizable loop with dependencies

- Patterns that can be vectorized despite “dependencies”
 - **Reduction loops** are often auto-vectorized by modern compilers. For floating-point, vectorization may require relaxed FP semantics (e.g., fast-math), because it changes the summation order.
 - Some operators are commutative and associative for integers, not entirely for floating points
 - Compiler can use partial values and combine them at the end
 - Typical reduction operators: +, *, min, max, &, |, ^

```
for (auto i = 0; i < N; i++)  
    sum += A[i];
```

NOTE: With floating-point arithmetic, addition is not associative. Therefore, compiler optimizations such as reassociation or vectorized reductions, especially with `-ffast-math`, may produce slightly different results, the reduction sum may be different.

```
// Conceptual vectorization  
__m256 vsum = _mm256_setzero_ps();  
for (int i = 0; i < N; i += 8) {  
    __m256 v = _mm256_load_ps(&A[i]);  
    vsum     = _mm256_add_ps(vsum, v);  
}  
sum = hsum_avx(vsum);
```

Transformations of loops

- When dependencies can be removed, two examples:

```
// original (WAR dependencies)
for (int_i=1; i<N-1; ++i) {
    B[i] = A[i];
    A[i] = A[i+1];
}
```



```
// loop fission, both loops are
vectorizable
for (int_i=1; i<N-1; ++i) B[i] = A[i];
for (int_i=1; i<N-1; ++i) A[i] = A[i+1];
```

```
// inner loop has RAW dependency (sum)
for (int_i=0; i<M; ++i) {
    float sum = 0.0f;
    for (int_j=0; j<N; ++j) sum+=B[j][i];
    A[i] = sum;
}
```



```
// loop interchange: inner loop vectorizable
for (int_i=0; i<M; ++i) A[i]=0.0f;
for (int_j=0; j<N; ++j) {
    for (int_i=0; i<M; ++i)
        A[i] += B[j][i];
}
```


Aliasing

- Auto-vectorization is often blocked by potential aliasing
 - Two (or more) pointers (or references) point to overlapping regions of memory
 - Compiler is conservative: assume aliasing unless proved otherwise
- Runtime checks are a strategy that compilers can use to enable vectorization
- Use compiler extension **qualifier**, e.g., `__restrict__` in C++ to tell the compiler that there is no aliasing

```
// If you know there are no aliasing issues
void add(float* __restrict__ a,
        const float* __restrict__ b, size_t n) {
    for (size_t i = 0; i < n; ++i)
        a[i] += b[i];
}
```

```
// potential aliasing for a and b
void add(float* a, float* b, size_t n) {
    for (size_t i = 0; i < n; i++)
        a[i] += b[i];
}
```

```
// runtime check to enable vectorization
if ( a + n < b || b + n < a ) {
    // a and b do not overlap
    // vectorized code
} else {
    for (size_t i = 0; i < n; i++)
        a[i] += b[i];
}
```

```
// C++ references don't alias by default (language rule)
void add(std::vector<float>& a, std::vector<float>& b) {
    for (size_t i = 0; i < a.size(); ++i)
        a[i] += b[i];
}
```

Code Style

- When compilers try to convert scalar code to vector operations automatically, one major challenge is how the data is organized in memory

- Prefer SoA to AoS**

- SoA makes it easier for compilers to perform vectorized loads/stores
- Converting AoS to SoA might require restructuring the code and data, which can be non-trivial

- Prefer unit stride access in the innermost loop** (e.g., i,k,j instead of i,j,k in GEMM)

- Efficient prefetching of data
- Loading of vector registers with fewer instructions
- Better cache locality

What is **stride**? Distance between consecutive memory accesses

Non-unit stride = $k > 1$ ($a[i]$, $a[i+k]$, $a[i+2k]$, ...)

```
//Matrix Multiply with unit stride
for (int i=0; i<N; i++)
  for (int k=0; k<L; k++) {
    float Aik = A[i*L+k]; // reused N times
    for (int j=0; j<M; j++)
      C[i*M+j] += Aik * B[k*M+j];
  }
```

Execution on the spmcluster using $m = n = 1 = 2^{10}$

#elapsed time (naive_mult): 2.44s

#elapsed time (looporder_mult): 0.16s

Compilation flags: `-O3 -march=native -mavx2 -ffast-math`

General tips for loop vectorization

- The loop count must be known at entry to the loop
 - The variable setting the number of iterations must remain constant for the duration of the loop
- The exit of the loop must not be data-dependent. Avoid conditional `'break'`
- Avoid `'switch'`/`'return'` statements inside the loop. The `'if'` statement can be vectorized if it can be implemented as a masked assignment
- No function calls. If a function call can be inlined, this is okay. Intrinsics math functions (e.g., `sqrt`) are allowed. Library functions inside the loop body may prevent vectorization
- Use aligned addresses to avoid the compiler peeling the loop to align memory accesses
 - If unaligned, the compiler can generate extra code (peeling) to handle misaligned data at the beginning/end of a loop
- Avoid dependencies between loop iterations, if possible
- Avoid pointer aliasing by using `__restrict`

Simple code examples

- Let's see the four simple examples in [auto-vect-simple.zip](#)
- Each file highlights one common factor that enables or blocks auto-vectorization (unit-stride, function calls, external calls, irregular access patterns)
- Build with GCC reports: `-fopt-info-vec-optimized` `-fopt-info-vec-missed` and look for “loop vectorized” vs “couldn't vectorize loop”
 - `one.cpp`: canonical unit-stride loop, expected “loop vectorized using 32 byte vectors” (AVX2)
 - `two.cpp`: inline vs `noinline` call, inline loop often vectorized, `noinline` often “control flow in loop” or “can't vectorize”.
 - `three.cpp`: call from shared library, expected “not vectorized” due to external call (often flagged as “control flow” and “can throw exception”)
 - `not-vectorized.cpp`: irregular access (indirect indexing / non unit-stride), expected “complicated access pattern” or “missed”.
 - “missed: clobbers memory” often comes from I/O or timers, not from the loop itself

Matrix Multiplication, auto-vectorization

SPM cluster (front-end node)
Intel(R) Xeon(R) CPU
E5-2650 v3 @ 2.30GHz

using $m = n = 2^{13}, l = 2^8$

#elapsed time (intrinsic): 3.22s

#elapsed time (auto-vectorization): 3.12s

```
[torquati@spmln spmcode2]$ g++ -std=c++17 -I. -I./include -Wall -march=native -O3 -o plain tmm_avx.cpp
[torquati@spmln spmcode2]$ g++ -std=c++17 -I. -I./include -Wall -march=native -ffast-math -O3 -o auto tmm_avx.cpp
[torquati@spmln spmcode2]$ g++ -std=c++17 -I. -I./include -Wall -march=native -ffast-math -O3 -o intrinsics tmm_avx.cpp
[torquati@spmln spmcode2]$ ./plain 13 13 8
# elapsed time (alloc_memory): 2.6236e-05 s
# elapsed time (init): 0.0209272 s
# elapsed time (plain_tmm): 17.0662 s
[torquati@spmln spmcode2]$ ./auto 13 13 8
# elapsed time (alloc_memory): 2.7676e-05 s
# elapsed time (init): 0.0189545 s
# elapsed time (plain_tmm): 3.1178 s
[torquati@spmln spmcode2]$ ./intrinsics 13 13 8
# elapsed time (alloc_memory): 2.8324e-05 s
# elapsed time (init): 0.0190705 s
# elapsed time (avx_tmm): 3.22404 s
[torquati@spmln spmcode2]$
```

Loop unrolling for max reduction

- **Max reduction**: the *plain_max* function computes the maximum of a given array of floating-point numbers.
- Compilers may not auto-vectorize this loop because of NaN semantics and the reduction pattern (depends on flags (e.g., -ffast-math), and inlining)

```
float plain_max(float * data, uint64_t length) {  
    float max=-std::numeric_limits<float>::infinity();  
    for (uint64_t i = 0; i < length; i++)  
        max = std::max(max, data[i]);  
    return max;  
}
```

What do you expect if we unroll the loop by a factor of 2?

- More ILP
- Less loop overhead (fewer branches)

```
// assume length is even  
float plain_max_unroll_2(float * data, uint64_t length) {  
    float max_0 =-std::numeric_limits<float>::infinity();  
    float max_1 =-std::numeric_limits<float>::infinity();  
    for (uint64_t i = 0; i < length; i+=2) {  
        max_0 = std::max(max_0, data[i+0]);  
        max_1 = std::max(max_1, data[i+1]);  
    }  
    return std::max(max_0, max_1);  
}
```


Loop unrolling (cont.)

Loop Unrolling for Max Reduction

```
[torquati@spmln spmcode2]$ ./plain_vector_max
2.68435e+08
# elapsed time (plain_max): 0.3284 s
2.68435e+08
# elapsed time (plain_max_unroll_2): 0.180033 s
2.68435e+08
# elapsed time (plain_max_unroll_4): 0.13654 s
2.68435e+08
# elapsed time (plain_max_unroll_8): 0.135766 s
2.68435e+08
# elapsed time (avx_max): 0.0917001 s
```

Why do they run faster?

- Reduced loop overhead, i.e., **fewer control instructions are executed**
- Increased ILP and better pipelining, max_0/1/2/3 can be computed independently
- Fewer RAW dependencies because of independent max_ variables

Beyond a certain factor, excessive loop unrolling can hurt performance.

- However, loop unrolling may not always result in performance improvements, as it can increase code size and register pressure, which can have negative effects on performance in some cases

```
float plain_max_unroll_4(float * data, uint64_t length) {
    float max_0 = -INFINITY, max_1 = -INFINITY;
    float max_2 = -INFINITY, max_3 = -INFINITY;
    for (uint64_t i = 0; i < length; i+=4) {
        max_0 = std::max(max_0, data[i+0]);
        max_1 = std::max(max_1, data[i+1]);
        max_2 = std::max(max_2, data[i+2]);
        max_3 = std::max(max_3, data[i+3]);
    }
    return std::max(max_0, std::max(max_1,
                                    std::max(max_2, max_3)));
}
```

Compiler pragma:

- **#pragma GCC unroll 2**
- It may still ignore it!

Suggested Readings

- Chapter 3 of the Parallel Programming Concept and Practice book
- Take a look at the examples code provided (e.g., timing measurements). Try to modify the code and play with compiler options for auto-vectorization of the code.
- The following online document considers auto-vectorization using the Intel compiler. However, it contains general guidelines valid also for the GCC compiler
<https://www.intel.com/content/dam/develop/external/us/en/documents/31848-compilerautovectorizationguide.pdf>